Complexity and Recurrences

Algorithmic Performance and Mathematical Analysis

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-I

Course Outline

1. Introduction to Complexity Analysis
2. Advanced Complexity Analysis

Outline

1 Introduction to Complexity Analysis

2 Advanced Complexity Analysis

Why Analyze Algorithms?

The Fundamental Question

Given multiple algorithms that solve the same problem, which one should we choose?

- **Performance:** How fast does the algorithm run?
- **Resources:** How much memory does it use?
- **Scalability:** How does it handle large inputs?
- **Predictability:** Can we predict its behavior?

Real-World Impact

- User experience depends on algorithm speed
- Server costs increase with inefficient algorithms
- Mobile apps drain battery with poor algorithms
- Scientific computing requires efficient solutions

Why Analyze Algorithms?

The Fundamental Question

Given multiple algorithms that solve the same problem, which one should we choose?

- **Performance:** How fast does the algorithm run?
- **Resources:** How much memory does it use?
- **Scalability:** How does it handle large inputs?
- **Predictability:** Can we predict its behavior?

Real-World Impact

- User experience depends on algorithm speed
- Server costs increase with inefficient algorithms
- Mobile apps drain battery with poor algorithms
- Scientific computing requires efficient solutions

Why Analyze Algorithms?

The Fundamental Question

Given multiple algorithms that solve the same problem, which one should we choose?

- **Performance:** How fast does the algorithm run?
- **Resources:** How much memory does it use?
- **Scalability:** How does it handle large inputs?
- **Predictability:** Can we predict its behavior?

Real-World Impact

- User experience depends on algorithm speed
- Server costs increase with inefficient algorithms
- Mobile apps drain battery with poor algorithms
- Scientific computing requires efficient solutions

Elementary Operations Identification I

Definition

An **elementary operation** is a basic computation that takes constant time regardless of input size.

Common Elementary Operations:

- **Assignment:** `x = 5`
- **Arithmetic:** $+, -, *, /$
- **Comparisons:** $=, <, >$
- **Array Access:** `A[i]`
- **Function Calls:** Simple built-in functions

Elementary Operations Identification I

Definition

An **elementary operation** is a basic computation that takes constant time regardless of input size.

Common Elementary Operations:

- **Assignment:** `x = 5`
- **Arithmetic:** `+`, `-`, `*`, `/`
- **Comparison:** `==`, `<`, `>`
- **Array Access:** `array[i]`
- **Function Calls:** Simple built-in functions

Elementary Operations Identification I

Definition

An **elementary operation** is a basic computation that takes constant time regardless of input size.

Common Elementary Operations:

- **Assignment:** `x = 5`
- **Arithmetic:** `+`, `-`, `*`, `/`
- **Comparison:** `==`, `<`, `>`
- **Array Access:** `array[i]`
- **Function Calls:** Simple built-in functions

Elementary Operations Identification I

Definition

An **elementary operation** is a basic computation that takes constant time regardless of input size.

Common Elementary Operations:

- **Assignment:** `x = 5`
- **Arithmetic:** `+`, `-`, `*`, `/`
- **Comparison:** `==`, `<`, `>`
- **Array Access:** `array[i]`
- **Function Calls:** Simple built-in functions

Elementary Operations Identification I

Definition

An **elementary operation** is a basic computation that takes constant time regardless of input size.

Common Elementary Operations:

- **Assignment:** `x = 5`
- **Arithmetic:** `+`, `-`, `*`, `/`
- **Comparison:** `==`, `<`, `>`
- **Array Access:** `array[i]`
- **Function Calls:** Simple built-in functions

Elementary Operations Identification I

Definition

An **elementary operation** is a basic computation that takes constant time regardless of input size.

Common Elementary Operations:

- **Assignment:** `x = 5`
- **Arithmetic:** `+`, `-`, `*`, `/`
- **Comparison:** `==`, `<`, `>`
- **Array Access:** `array[i]`
- **Function Calls:** Simple built-in functions

Elementary Operations Identification II

Example

Count operations in simple code:

```
sum = 0 // 1 operation
for i = 0 to n − 1 do
    sum = sum + array[i] // 2 operations per iteration
end for
```

Total: $1 + 2n$ operations

Study Case: Counting Operations in Arrays Union I

Problem

Given two arrays A and B, create array C that contains all elements from both arrays without duplicates.

```
Copy all elements from A to C
for each element in B do
    Check if element exists in C
    if element not found then
        Add element to C
    end if
end for
```

Study Case: Counting Operations in Arrays Union I

Problem

Given two arrays A and B, create array C that contains all elements from both arrays without duplicates.

```
Copy all elements from A to C
for each element in B do
    Check if element exists in C
    if element not found then
        Add element to C
    end if
end for
```

Study Case: Counting Operations in Arrays Union II

Operation Analysis

- **Copy from A:** n operations
- **For each element in B:** m iterations
- **Search in C:** Up to n comparisons per element
- **Total worst case:** $n + m * n = n(1 + m)$ operations

Key Insight

The nested search makes this algorithm slow for large arrays! Performance grows quadratically.

Study Case: Counting Operations in Arrays Union II

Operation Analysis

- **Copy from A:** n operations
- **For each element in B:** m iterations
- **Search in C:** Up to n comparisons per element
- **Total worst case:** $n + m * n = n(1 + m)$ operations

Key Insight

The nested search makes this algorithm slow for large arrays! Performance grows quadratically.

Best, Average, Worst Case Scenarios

Three Perspectives on Algorithm Performance

Different inputs can lead to dramatically different execution times for the `same algorithm`.

Best Case

- Minimum operations
- Most favorable input
- Example: Finding element at first position

Average Case

- Expected performance
- Typical input
- Example: Finding element in middle

Worst Case

- Maximum operations
- Least favorable input
- Example: Element not found

Best, Average, Worst Case Scenarios

Three Perspectives on Algorithm Performance

Different inputs can lead to dramatically different execution times for the `same algorithm`.

Best Case

- Minimum operations
- Most favorable input
- Example: Finding element at first position

Average Case

- Expected performance
- Typical input
- Example: Finding element in middle

Worst Case

- Maximum operations
- Least favorable input
- Example: Element not found

Best, Average, Worst Case Scenarios

Three Perspectives on Algorithm Performance

Different inputs can lead to dramatically different execution times for the `same algorithm`.

Best Case

- Minimum operations
- Most favorable input
- Example: Finding element at first position

Average Case

- Expected performance
- Typical input
- Example: Finding element in middle

Worst Case

- Maximum operations
- Least favorable input
- Example: Element not found

Best, Average, Worst Case Scenarios

Three Perspectives on Algorithm Performance

Different inputs can lead to dramatically different execution times for the `same algorithm`.

Best Case

- Minimum operations
- Most favorable input
- Example: Finding element at first position

Average Case

- Expected performance
- Typical input
- Example: Finding element in middle

Worst Case

- Maximum operations
- Least favorable input
- Example: Element not found

Best, Average, Worst Case Scenarios II

Example

Search in array [5, 2, 8, 1, 9]:

- **Best:** Search for $5 \rightarrow 1$ comparison
- **Average:** Random element $\rightarrow\sim 2.5$ comparisons
- **Worst:** Search for 7(not found) $\rightarrow 5$ comparisons

Performance Prediction Methods

Approaches to Predict Algorithm Performance

1. **Theoretical Analysis:** Mathematical analysis
2. **Empirical Analysis:** Timing actual executions
3. **Simulation:** Testing with synthetic data
4. **Profiling:** Detailed runtime analysis

Performance Prediction Methods

Approaches to Predict Algorithm Performance

1. **Theoretical Analysis:** Mathematical analysis
2. **Empirical Analysis:** Timing actual executions
3. **Simulation:** Testing with synthetic data
4. **Profiling:** Detailed runtime analysis

Performance Prediction Methods

Approaches to Predict Algorithm Performance

1. **Theoretical Analysis:** Mathematical analysis
2. **Empirical Analysis:** Timing actual executions
3. **Simulation:** Testing with synthetic data
4. **Profiling:** Detailed runtime analysis

Performance Prediction Methods

Approaches to Predict Algorithm Performance

1. **Theoretical Analysis:** Mathematical analysis
2. **Empirical Analysis:** Timing actual executions
3. **Simulation:** Testing with synthetic data
4. **Profiling:** Detailed runtime analysis

Performance Prediction Methods II

Theoretical Analysis

Advantages:

- Machine-independent
- Covers all possible inputs
- Mathematical precision

Limitations:

- Ignores implementation details
- May not reflect real hardware

Empirical Analysis

Advantages:

- Reflects real performance
- Shows actual constants
- Platform-specific results

Limitations:

- Machine-dependent
- Limited input coverage
- Time-consuming

Best Practice

Use theoretical analysis for algorithm design, then validate with empirical testing.

Performance Prediction Methods II

Theoretical Analysis

Advantages:

- Machine-independent
- Covers all possible inputs
- Mathematical precision

Limitations:

- Ignores implementation details
- May not reflect real hardware

Empirical Analysis

Advantages:

- Reflects real performance
- Shows actual constants
- Platform-specific results

Limitations:

- Machine-dependent
- Limited input coverage
- Time-consuming

Best Practice

Use theoretical analysis for algorithm design, then validate with empirical testing.

Performance Prediction Methods II

Theoretical Analysis

Advantages:

- Machine-independent
- Covers all possible inputs
- Mathematical precision

Limitations:

- Ignores implementation details
- May not reflect real hardware

Empirical Analysis

Advantages:

- Reflects real performance
- Shows actual constants
- Platform-specific results

Limitations:

- Machine-dependent
- Limited input coverage
- Time-consuming

Best Practice

Use theoretical analysis for algorithm design, then validate with empirical testing.

Time vs. Space Trade-offs I

Definition

A time-space trade-off occurs when we can reduce time by using more memory, or reduce memory at the cost of increased time.

Trade-off Decision Factors:

- Available memory constraints
- Time requirements
- Cost of computation vs. storage
- Frequency of algorithm execution

Time vs. Space Trade-offs I

Definition

A time-space trade-off occurs when we can reduce time by using more memory, or reduce memory at the cost of increased time.

Trade-off Decision Factors:

- Available memory constraints
- Time requirements
- Cost of computation vs. storage
- Frequency of algorithm execution

Time vs. Space Trade-offs II

Example

Fibonacci Calculation:

Space-Efficient (Recursive):

- Uses minimal extra memory
- But recalculates same values repeatedly
- Very slow for large numbers

Time-Efficient (Memoization):

- Stores previously calculated values
- Uses more memory
- Much faster for large numbers

Systematic Complexity Calculation Methods I

Step-by-Step Approach

1. **Identify:** All loops and recursive calls
2. **Count:** Operations in each section
3. **Multiply:** Operations by execution frequency
4. **Sum:** All contributions
5. **Simplify:** Focus on dominant terms

Systematic Complexity Calculation Methods II

Example

Nested Loop Analysis:

```
for i = 0 to n − 1 do
    for j = 0 to n − 1 do
        result = array[i] + array[j]
    end for
end for
```

Operation Count:

- Outer loop runs n times
- Inner loop runs n times for each outer iteration
- Total operations: $n * n = n^2$ operations

Study Case: Complexity Calculation for `QuickSort` I

QuickSort Algorithm Structure

1: Choose a pivot element
2: Partition array around pivot
3: Recursively sort left part
4: Recursively sort right part

Partition Operation

- Compare each element with pivot
- Rearrange elements (smaller left, larger right)
- This takes n operations for n elements

Study Case: Complexity Calculation for `QuickSort` I

QuickSort Algorithm Structure

1: Choose a pivot element
2: Partition array around pivot
3: Recursively sort left part
4: Recursively sort right part

Partition Operation

- Compare each element with pivot
- Rearrange elements (smaller left, larger right)
- This takes n operations for n elements

Study Case: Complexity Calculation for `QuickSort` II

Best Case

- Pivot divides array in half
- Balanced partitions
- Fast performance

Worst Case

- Pivot is always smallest/largest
- Unbalanced partitions
- Slow performance

Outline

1 Introduction to Complexity Analysis

2 Advanced Complexity Analysis

Big O Notation: Definition and Meaning

Definition

Big O notation describes the upper bound of an algorithm's growth rate for large inputs.

What Big O Tells Us:

- How algorithm performance scales with input size
- Worst-case growth rate
- Ignores constant factors and smaller terms
- Focuses on dominant behavior for large inputs

Big O Notation: Definition and Meaning

Definition

Big O notation describes the upper bound of an algorithm's growth rate for large inputs.

What Big O Tells Us:

- How algorithm performance scales with input size
- Worst-case growth rate
- Ignores constant factors and smaller terms
- Focuses on dominant behavior for large inputs

Big O Notation: Classifications

Common Big O Classifications:

- $O(1)$ — Constant: Always same time
- $O(n)$ — Linear: Time grows with input size
- $O(n^2)$ — Quadratic: Time grows with square of input
- $O(logn)$ — Logarithmic: Very efficient growth

Key Insight

Big O focuses on asymptotic behavior — how algorithms scale with very large inputs.

Big O Notation: Classifications

Common Big O Classifications:

- $O(1)$ — Constant: Always same time
- $O(n)$ — Linear: Time grows with input size
- $O(n^2)$ — Quadratic: Time grows with square of input
- $O(logn)$ — Logarithmic: Very efficient growth

Key Insight

Big O focuses on asymptotic behavior — how algorithms scale with very large inputs.

Upper Bound Concept and Mathematical Definition

Understanding Upper Bounds

An **upper bound** guarantees that an algorithm will never perform worse than the bound.

Big O Meaning:

- Sets a ceiling on performance
- Guarantees maximum time
- Used for worst-case analysis
- Helps compare algorithms

Common Growth Orders:

- $O(1)$ — Constant
- $O(logn)$ — Logarithmic
- $O(n)$ — Linear
- $O(n \log n)$ — Linearithmic
- $O(n^2)$ — Quadratic
- $O(2^n)$ — Exponential

Study Case: First Letter in a Sentence Complexity

Problem

Find the first alphabetical letter in a given sentence string.

```
for each character in sentence do
    if character is a letter then
        return character
    end if
end for
return null
```

Best: First character is a letter

- Only check 1 character
- Constant time: $O(1)$

Worst: No letters in sentence

- Check every character
- Linear time: $O(n)$

Even though best case is $O(1)$, we report $O(n)$ because Big O represents the worst-case upper bound.

Study Case: First Letter in a Sentence Complexity

Problem

Find the first alphabetical letter in a given sentence string.

```
for each character in sentence do
    if character is a letter then
        return character
    end if
end for
return null
```

Best: First character is a letter

- Only check 1 character
- Constant time: $O(1)$

Worst: No letters in sentence

- Check every character
- Linear time: $O(n)$

Even though best case is $O(1)$, we report $O(n)$ because Big O represents the worst-case upper bound.

Study Case: First Letter in a Sentence Complexity

Problem

Find the first alphabetical letter in a given sentence string.

```
for each character in sentence do
    if character is a letter then
        return character
    end if
end for
return null
```

Best: First character is a letter

- Only check 1 character
- Constant time: $O(1)$

Worst: No letters in sentence

- Check every character
- Linear time: $O(n)$

Even though best case is $O(1)$, we report $O(n)$ because Big O represents the worst-case upper bound.

Study Case: Linear Search Algorithm Complexity

Problem

Search for a target element in an unsorted array.

```
for each element in array do
    if element equals target then
        return index
    end if
end for
return -1
```

- **Best Case:** Element at first position, 1 comparison $\rightarrow O(1)$
- **Worst Case:** Element not in array, n comparisons $\rightarrow O(n)$
- **Average Case:** Element in middle, $n/2$ comparisons $\rightarrow O(n)$

Study Case: Linear Search Algorithm Complexity

Problem

Search for a target element in an unsorted array.

```
for each element in array do
    if element equals target then
        return index
    end if
end for
return -1
```

- **Best Case:** Element at first position, 1 comparison $\rightarrow O(1)$
- **Worst Case:** Element not in array, n comparisons $\rightarrow O(n)$
- **Average Case:** Element in middle, $n/2$ comparisons $\rightarrow O(n)$

Study Case: Binary Search Algorithm Complexity I

Problem

Search for a target element in a sorted array.

```
while search space not empty do
    Find middle element
    if middle equals target then
        return index
    else if middle ¡ target then
        search right half
    else
        search left half
    end if
end while
return -1
```

Study Case: Binary Search Algorithm Complexity II

Each step eliminates half of the remaining elements:

- Step 1: n elements
- Step 2: $n/2$ elements
- Step 3: $n/4$ elements
- Step k: $n/2^k$ elements

Algorithm stops when we reach 1 element.

Big Omega (Ω) and Big Theta (Θ) Notations I

Big Omega (Ω)

Lower Bound:

- Minimum time required
- Best-case analysis
- "At least this fast"
- Guarantees algorithm won't be faster

Big Theta (Θ)

Tight Bound:

- Exact growth rate
- Best and worst are same
- "Exactly this fast"
- Most precise description

Big Omega (Ω) and Big Theta (Θ) Notations II

Example

Merge Sort:

- Best case: $\Theta(n \log n)$
- Worst case: $O(n \log n)$
- Since best = worst: $\Theta(n \log n)$

Linear Search:

- Best case: $\Omega(1)$
- Worst case: $O(n)$
- No tight bound (best $\neq$ worst)

Study Case: Tower of Hanoi Problem I

Problem

Move n disks from source rod to destination rod using auxiliary rod. Rules: only move one disk at a time, never place larger disk on smaller one.

```
function Hanoi(n, source, destination, auxiliary)
if n = 1 then
    move disk from source to destination
else
    Hanoi(n − 1, source, auxiliary, destination)
    move disk from source to destination
    Hanoi(n − 1, auxiliary, destination, source)
end if
```

Study Case: Tower of Hanoi Problem I

Problem

Move n disks from source rod to destination rod using auxiliary rod. Rules: only move one disk at a time, never place larger disk on smaller one.

```
function Hanoi(n, source, destination, auxiliary)
if n = 1 then
    move disk from source to destination
else
    Hanoi(n − 1, source, auxiliary, destination)
    move disk from source to destination
    Hanoi(n − 1, auxiliary, destination, source)
end if
```

Study Case: Tower of Hanoi Problem II

Recurrence Analysis

- $T(1) = 1$ (base case: move one disk)
- $T(n) = 2T(n-1) + 1$ (two recursive calls + one move)

Solution:

- $T(n) = 2^n - 1$
- Complexity: $O(2^n)$ — exponential!

Real-World Implication

For 64 disks: $2^{64} - 1 \approx$ 18 billion billion moves!

Study Case: Tower of Hanoi Problem II

Recurrence Analysis

- $T(1) = 1$ (base case: move one disk)
- $T(n) = 2T(n-1) + 1$ (two recursive calls + one move)

Solution:

- $T(n) = 2^n - 1$
- Complexity: $O(2^n)$ — exponential!

Real-World Implication

For 64 disks: $2^{64} - 1 \approx$ 18 billion billion moves!

Solving Recurrence Relations

Common Solution Methods

1. **Substitution Method:** Guess and verify
2. **Recursion Tree:** Visualize recursive calls
3. **Master Theorem:** Direct formula for common cases
4. **Characteristic Equation:** For homogeneous recurrences

Solving Recurrence Relations

Example

Recursion Tree Method for $T(n) = 2T(n/2) + n$:

- Level 0: n work
- Level 1: $2 \times (n/2) = n$ work
- Level 2: $4 \times (n/4) = n$ work
- ...
- Total levels: $\log n$
- Total work: $n \times \log n = O(n \log n)$

Method Selection

- Simple cases: Direct solving
- Divide-and-conquer: Master theorem
- Complex cases: Substitution method

Solving Recurrence Relations

Example

Recursion Tree Method for $T(n) = 2T(n/2) + n$:

- Level 0: n work
- Level 1: $2 \times (n/2) = n$ work
- Level 2: $4 \times (n/4) = n$ work
- ...
- Total levels: $\log n$
- Total work: $n \times \log n = O(n \log n)$

Method Selection

- Simple cases: Direct solving
- Divide-and-conquer: Master theorem
- Complex cases: Substitution method

Master Theorem Introduction I

Master Theorem

For recurrences of the form $T(n) = aT(n/b) + f(n)$:

Case 1: $f(n)$ grows slower than $n^{(\log_b a)}$

- $T(n) = O(n^{(\log_b a)})$

Case 2: $f(n)$ grows at same rate

- $T(n) = O(n^{(\log_b a)} \log n)$

Case 3: $f(n)$ grows faster

- $T(n) = O(f(n))$

Master Theorem Introduction II

Examples

Merge Sort:

- $T(n) = 2T(n/2) + n$
- $a = 2,\ b = 2,\ f(n) = n$
- Case 2: $O(n \log n)$

Binary Search:

- $T(n) = T(n/2) + 1$
- $a = 1,\ b = 2,\ f(n) = 1$
- Case 2: $O(\log n)$

When to Use

Master theorem works for many divide-and-conquer algorithms but has limitations.

Master Theorem Introduction II

Examples

Merge Sort:

- $T(n) = 2T(n/2) + n$
- $a = 2,\ b = 2,\ f(n) = n$
- Case 2: $O(n \log n)$

Binary Search:

- $T(n) = T(n/2) + 1$
- $a = 1,\ b = 2,\ f(n) = 1$
- Case 2: $O(\log n)$

When to Use

Master theorem works for many divide-and-conquer algorithms but has limitations.

Recursive Algorithm Complexity Calculation I

Systematic Approach

1. **Identify:** Base case(s) and recursive case(s)
2. **Set up:** Recurrence relation
3. **Solve:** Using appropriate method
4. **Verify:** Check solution makes sense

Recursive Algorithm Complexity Calculation II

Merge Sort Analysis

Step 1: Identify cases

- Base: Array size $1 \to O(1)$
- Recursive: Divide, sort halves, merge

Step 2: Set up recurrence

- $T(n) = 2T(n/2) + O(n)$

Step 3: Solve using Master Theorem

- Case 2: $T(n) = O(n \log n)$

Step 4: Verify

- Makes sense: logarithmic levels, linear work per level

Summary I

Key Takeaways

- **Complexity Analysis:** Essential for algorithm comparison and selection
- **Big O Notation:** Describes growth rate for large inputs
- **Recursion:** Powerful technique requiring careful base case design
- **Recurrence Relations:** Mathematical tools for recursive algorithm analysis

Summary II

Practical Skills Gained

- Count operations systematically
- Classify algorithms by growth rate
- Design recursive solutions correctly
- Analyze recursive algorithm complexity
- Apply Master Theorem to common patterns

Next Steps

- Practice complexity analysis on various algorithms
- Implement recursive solutions with proper optimization
- Study advanced recurrence solving techniques
- Apply knowledge to real-world algorithm selection

Summary II

Practical Skills Gained

- Count operations systematically
- Classify algorithms by growth rate
- Design recursive solutions correctly
- Analyze recursive algorithm complexity
- Apply Master Theorem to common patterns

Next Steps

- Practice complexity analysis on various algorithms
- Implement recursive solutions with proper optimization
- Study advanced recurrence solving techniques
- Apply knowledge to real-world algorithm selection

Outline

1 Introduction to Complexity Analysis

2 Advanced Complexity Analysis

Thanks!

Questions?

Repo: *https://github.com/EngAndres/ud-public/tree/main/courses/computer-sciences-i*